

LC 904 → Fruits into Baskets

Deutsche Bank

Amazon

Problem Understanding

You have **2 baskets** and each basket can hold a **single type of fruit only**.

Hence, a window is considered valid if:

Distinct fruit types ≤ 2

1. Brute Force Approach ($O(N^2)$)

The naive approach is to check every possible contiguous subarray (window) and find the maximum length window that satisfies the condition.

```
for (int i = 0; i < n; i++) {
    unordered_map<int, int> freq;
    for (int j = i; j < n; j++) {
        freq[fruits[j]]++;

        // As soon as more than 2 fruit types arrive
        if (freq.size() > 2) {
            break;
        }

        ans = max(ans, j - i + 1);
    }
}
return ans;
```

2. Sliding Window Version (O(N))

To optimize the solution, we can use a sliding window (two pointers) approach to maintain a valid window where the number of distinct elements is at most 2.

Algorithm logic from notes:

```
while (r < n) {
    freq[fruits[r]]++;
    while (freq.size() > 2) {
        freq[fruits[l]]--;
        if (freq[fruits[l]] == 0)
            freq.erase(fruits[l]);
        l++;
    }
    ans = max(ans, r - l + 1);
    r++;
}
```

Complete C++ Solution

Below is the fully formatted C++ code provided for the optimal sliding window approach.

```
class Solution {
public:
    int totalFruit(vector<int>& fruits) {
        int n = fruits.size();
        unordered_map<int, int> freq;
        int l = 0;
        int r = 0;
        int ans = 0;

        while(r < n) {
            freq[fruits[r]]++;

            while(freq.size() > 2) {
                freq[fruits[l]]--;
                if(freq[fruits[l]] == 0) {
                    freq.erase(fruits[l]);
                }
                l++;
            }

            ans = max(ans, r - l + 1);
            r++;
        }

        return ans;
    }
};
```